

VIBE-CODED APP SECURITY

PART 3 / 4 • CHECKS 01-54 • @byarnieverma

The practical launch filter for AI-built apps. Your app can look finished and still be unsafe.

SERIES BUILD

01 COMPLETE

02 COMPLETE

03 CURRENT

04 LOCKED

THE RULE If you cannot point to the code, setting, configuration, test or log that proves a guardrail exists, treat it as missing.

HOW TO USE THIS

01 AUDIT

Run the prompt

02 FIX

FAIL + UNKNOWN

03 VERIFY

Test the deployed path

04 RE-RUN

After major changes

COPY / PASTE AUDIT PROMPT

Act as a security reviewer for this application. Audit the repository and deployed configuration against checks 1-54 in this resource. For every relevant item return PASS, FAIL, UNKNOWN, or NOT APPLICABLE. Cite the exact file, configuration or setting supporting the result. Do not mark PASS without evidence. For FAIL or UNKNOWN, explain the realistic failure mode in one sentence, give the smallest safe fix, and tell me how to verify it. Prioritise anything touching authentication, authorization, private data, payments, admin access, secrets, AI tools or spend. Do not change production data or infrastructure during the audit.

PART 1 SECRETS, AUTHENTICATION & INPUT

Stop the obvious shortcuts AI-generated apps tend to ship with.

01 Exposed database credentials

Keep database usernames, passwords and connection strings server-side; rotate anything that was exposed.

03 Hardcoded API keys or secrets

Move private keys to server-side environment variables or a secret manager and rotate leaked values.

05 Missing server-side authorization

Check what the signed-in user is allowed to do on the server before every sensitive action.

07 Open database permissions

Default-deny database access and grant the application only the reads and writes it genuinely needs.

09 Unprotected admin routes

Enforce admin permissions on the server; a hidden button or secret URL is not an access control.

11 Build logs leaking secrets

Mask credentials in CI/CD and make sure scripts never print tokens, keys or connection strings.

13 Secrets in Git history

Treat a committed secret as exposed even after deletion; rotate it and remove it from history where appropriate.

15 Client-side-only security checks

Repeat validation, authorization and entitlement checks on the trusted server, not only in the UI.

02 Public .env files

Keep environment files out of Git, public builds and static hosting; use platform secret storage in production.

04 Weak or missing authentication

Use a proven auth system and require authentication on every route that should be private.

06 Cross-user data access

Scope reads and writes to the authenticated user or tenant so changing an ID cannot expose someone else's data.

08 Misconfigured Firebase / Supabase / S3

Review database and storage rules, then test them while signed out and as the wrong user.

10 Production debug tools exposed

Disable or strongly protect debug consoles, test routes, profilers and internal developer tools.

12 Verbose production errors

Return generic errors to users and keep stack traces, queries, paths and internal details in private logs.

14 Secrets shipped in frontend JavaScript

Anything sent to the browser is readable by users; private service credentials must stay server-side.

16 Missing input validation

Validate type, length, format, allowed values and size for every piece of untrusted input on the server.

17 SQL injection

Use parameterized queries or safe ORM bindings instead of concatenating user input into SQL.

18 NoSQL injection

Validate object shapes and operators and use safe query APIs so user-controlled objects cannot change query logic.

PART 2 WEB, SESSIONS, APIs & PAYMENTS

Protect the places users, browsers, files and money touch your backend.

19 Cross-site scripting (XSS)

Escape untrusted output, sanitize any allowed HTML and use a Content Security Policy where appropriate.

21 Insecure file uploads

Restrict file type and size, generate safe filenames, store files safely and scan risky uploads when appropriate.

23 Server-side request forgery (SSRF)

Allowlist external destinations where possible and block requests to private or internal network ranges.

25 Weak session management

Use strong session tokens, sensible expiry, rotation and server-side invalidation on logout or security changes.

27 Overly permissive CORS

Allow only the origins, methods, headers and credential combinations your application actually needs.

29 Unprotected staging or test environments

Authenticate non-production systems and keep production secrets, data and admin tools out of them.

31 Webhook signatures not verified

Verify the provider's webhook signature before trusting or processing the event.

33 IDOR / BOLA

Authorize the specific object being requested every time; possession of its ID must never be enough.

35 Sensitive data in logs

Redact passwords, tokens, payment data and unnecessary PII, then restrict log access and retention.

20 Cross-site request forgery (CSRF)

Use appropriate SameSite cookies and framework CSRF protections for browser-authenticated state changes.

22 Path traversal

Never trust a user-supplied path or filename; resolve access against an approved base directory.

24 Broken password-reset flows

Use short-lived, single-use reset tokens and avoid leaking whether a particular account exists.

26 Weak or incorrectly validated JWTs

Use strong signing keys and verify signature, issuer, audience, expiry and allowed algorithms.

28 Missing rate limits

Rate-limit login, signup, password reset, APIs and AI routes using sensible per-user and per-IP ceilings.

30 Default credentials left unchanged

Replace vendor defaults before deployment and remove unused default accounts or tokens.

32 Frontend-only payment checks

Determine subscription and entitlement state on the server rather than trusting browser state.

34 APIs trusting user-controlled roles or IDs

Derive identity and permissions from trusted server-side auth context rather than request fields.

36 Sensitive source maps or build artifacts

Inspect production output and exclude source maps or artifacts that reveal secrets or unintended internal files.

NEW IN PART 3 DEPENDENCIES, AI, DATA & INFRA

Cover the supply chain, AI permissions, observability and production data layer.

37 Vulnerable or abandoned dependencies

Scan dependencies, patch known vulnerabilities quickly and replace libraries that are no longer maintained.

39 Prompt injection

Separate trusted instructions from untrusted content and enforce permissions outside the model itself.

41 Excess database privileges

Give the application a least-privilege database role and isolate operations that genuinely need elevated access.

43 No security monitoring or alerts

Alert on auth abuse, privilege changes, unusual traffic, webhook failures, critical exceptions and spend spikes.

45 Public internal dashboards

Put admin, database, queue and monitoring dashboards behind strong authentication and network controls.

38 Malicious or compromised packages

Minimise dependencies, verify package identity and maintainers, and review suspicious install scripts.

40 AI tools bypassing user permissions

Authorize every tool call with the real user and tenant context before the model can access data or take action.

42 Missing audit logs

Record actor, action, target, time and outcome for sensitive changes so important activity can be reconstructed.

44 No tested backup / restore plan

Automate protected backups and prove that you can restore them before you rely on them.

46 Missing security headers

Configure browser protections such as CSP and anti-sniffing headers where relevant, then test the deployed response.

47 Unsafe cookie settings

Set HttpOnly, Secure and an appropriate SameSite policy on sensitive cookies based on how they are used.

49 Poor tenant isolation

Include tenant scope in authorization and data access at every layer of a multi-user or multi-organisation app.

51 Mass assignment / over-posting

Allowlist fields a user is allowed to update so they cannot submit hidden fields such as role, balance or ownership.

53 Unsafe deserialization

Use safe formats, strict schemas and integrity checks; do not deserialize attacker-controlled objects with unsafe mechanisms.

48 Sensitive data unprotected in transit / at rest

Use HTTPS/TLS, provider encryption and sensible key management for data that needs protection.

50 Over-trusting AI-generated code

Review diffs, run scanners and tests, and manually inspect auth, payment, data and permission logic before shipping.

52 Command / OS injection

Avoid shell execution where possible; otherwise use safe APIs and strictly validated arguments rather than concatenated commands.

54 Misconfigured OAuth / OIDC / social login

Restrict redirect URIs and correctly validate state or nonce, issuer, audience and token integrity.

OFFICIAL SOURCES

- [OWASP ASVS](#)
- [Stripe Webhooks](#)
- [OWASP Logging Cheat Sheet](#)
- [OWASP OAuth2 Cheat Sheet](#)
- [OWASP REST Security Cheat Sheet](#)
- [OWASP GenAI Security Project](#)
- [OWASP HTTP Headers Cheat Sheet](#)
- [OWASP Software Supply Chain Security](#)

NEXT RELEASE Part 4 adds checks 55–70: logic, CI/CD & advanced risks. This document is cumulative, so the next version includes everything here.

IMPORTANT Educational security checklist only. It is not a penetration test, legal advice, or a guarantee that an app cannot be compromised. High-risk apps and sensitive data deserve professional security review.

Save this. Part 4 expands the same launch filter.